

```

<namespace net.cn.nclzr.saktg;
?php

class AnomalyRecordManager
{
    private $pdo;

    /**
     * 构造函数，初始化PDO数据库连接
     */
    public function __construct()
    {
        // 使用PDO建立record库连接
        $this->pdo = new PDO('mysql:host=192.168.0.1;dbname=testdb', 'username',
'password');
        $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    }

    /**
     * 增加异常记录
     *
     * @param string $anomaly_type 异常类型
     * @param bigint $source_id 关联数据源ID
     * @param string $source_name 数据源名称
     * @param bigint $task_id 关联任务ID
     * @param string $task_name 任务名称
     * @param string $severity 严重程度
     * @param string $content 异常内容
     * @param string $status 处理状态
     * @return void
     */
    public function addAnomalyRecord($anomaly_type, $source_id, $source_name, $task_id,
$task_name, $severity, $content, $status)
    {
        // 准备SQL插入语句
        $sql = "INSERT INTO tb_anomaly_record (anomaly_type, source_id, source_name,
task_id, task_name, severity, content, status, occur_time)
            VALUES (:anomaly_type, :source_id, :source_name, :task_id, :task_name,
:severity, :content, :status, CURRENT_TIMESTAMP)";

        $stmt = $this->pdo->prepare($sql);
        // 绑定参数
        $stmt->bindParam(':anomaly_type', $anomaly_type);
        $stmt->bindParam(':source_id', $source_id, PDO::PARAM_INT);
        $stmt->bindParam(':source_name', $source_name);
        $stmt->bindParam(':task_id', $task_id, PDO::PARAM_INT);
        $stmt->bindParam(':task_name', $task_name);
        $stmt->bindParam(':severity', $severity);
        $stmt->bindParam(':content', $content);
        $stmt->bindParam(':status', $status);

        // 执插入操作
    }
}

```

```

        $stmt->execute();
    }

/**
 * 删除异常记录
 *
 * @param bigint $id 异常记录的ID
 * @return void
 */
public function deleteAnomalyRecord($id)
{
    // 准备SQL干掉语句
    $sql = "DELETE FROM tb_anomaly_record WHERE id = :id";
    $stmt = $this->pdo->prepare($sql);
    // 绑定参数
    $stmt->bindParam(':id', $id, PDO::PARAM_INT);

    // 执废弃操作
    $stmt->execute();
}

/**
 * 编辑异常记录
 *
 * @param bigint $id 异常记录ID
 * @param string $anomaly_type 异常类型
 * @param string $source_name 数据源名称
 * @param string $severity 严重程度
 * @param string $content 异常内容
 * @param string $status 处理状态
 * @return void
 */
public function editAnomalyRecord($id, $anomaly_type, $source_name, $severity,
$content, $status)
{
    // 准备SQL应用变更语句
    $sql = "UPDATE tb_anomaly_record
        SET anomaly_type = :anomaly_type, source_name = :source_name,
            severity = :severity, content = :content, status = :status,
process_time = CURRENT_TIMESTAMP
        WHERE id = :id";

    $stmt = $this->pdo->prepare($sql);

    // 绑定参数
    $stmt->bindParam(':anomaly_type', $anomaly_type);
    $stmt->bindParam(':source_name', $source_name);
    $stmt->bindParam(':severity', $severity);
    $stmt->bindParam(':content', $content);
    $stmt->bindParam(':status', $status);
    $stmt->bindParam(':id', $id, PDO::PARAM_INT);

```

```

        // 执变更操作
        $stmt->execute();
    }

/**
 * 查看异常记录详情
 *
 * @param bigint $id 异常记录ID
 * @return array
 */
public function viewAnomalyRecord($id)
{
    // 准备SQL选择语句
    $sql = "SELECT * FROM tb_anomaly_record WHERE id = :id";
    $stmt = $this->pdo->prepare($sql);

    // 绑定参数，并执
    $stmt->bindParam(':id', $id, PDO::PARAM_INT);
    $stmt->execute();

    // 返回查询结果
    return $stmt->fetch(PDO::FETCH_ASSOC);
}

/**
 * 分页查询异常记录
 *
 * @param int $page 当前页面
 * @param int $limit 每页记录条数
 * @return array 包含分页信息
 */
public function paginateAnomalyRecords($page, $limit)
{
    // 当前页面的起始位置计算
    $offset = ($page - 1) * $limit;

    // 查询写入条数
    $countSql = "SELECT COUNT(*) FROM tb_anomaly_record";
    $totalItems = $this->pdo->query($countSql)->fetchColumn();

    // 查询recordcollection
    $dataSql = "SELECT * FROM tb_anomaly_record LIMIT :offset, :limit";
    $stmt = $this->pdo->prepare($dataSql);
    $stmt->bindParam(':offset', $offset, PDO::PARAM_INT);
    $stmt->bindParam(':limit', $limit, PDO::PARAM_INT);

    // 执查询并返回结果
    $stmt->execute();
    $records = $stmt->fetchAll(PDO::FETCH_ASSOC);
}

```

```

        // 包装outputdata
        return [
            "data" => $records,
            "total_items" => $totalItems,
            "total_pages" => ceil($totalItems / $limit),
            "current_page" => $page
        ];
    }
}
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use MongoDB\Client as MongoClient;

/**
 * Class ExceptionEvent
 *
 * 负责管理系统异常检测记录，包括记录的增删查改和报告生成。
 */
class ExceptionEvent
{
    protected $mongoDB;

    public function __construct()
    {
        $this->mongoDB = (new MongoClient)->selectDatabase('exception_management');
    }

    /**
     * 在系统中记录异常事件。
     *
     * @param string $eventType 异常事件的类型
     * @param string $description 异常事件的详细描述
     * @param DateTime|null $occurredAt 异常事件发生的时间，默认为当前时间
     * @return void
     * @throws Exception 如果参数不合法，抛出异常
     */
    public function RecordExceptionEvent(string $eventType, string $description,
        DateTime $occurredAt = null): void
    {
        if (empty($eventType) || empty($description)) {
            throw new Exception('事件类型和描述不能为空', 400);
        }

        $eventData = [
            'eventType' => $eventType,
            'description' => $description,
            'occurredAt' => $occurredAt ? $occurredAt : new DateTime(),
            'status' => 'unresolved'
        ];

        $this->mongoDB->exception_records->insertOne($eventData);
    }
}

```

```

}

/**
 * 获取指定时间范围内的异常事件记录。
 *
 * @param DateTime $startDate 开始时间
 * @param DateTime $endDate 结束时间
 * @param array $filters 筛选条件
 * @return array 返回符合条件的异常事件记录
 * @throws Exception 如果日期范围不合法，抛出异常
 */
public function RetrieveExceptionEvents(DateTime $startDate, DateTime $endDate,
array $filters = []): array
{
    if ($startDate > $endDate) {
        throw new Exception(' 开始时间必须早于结束时间', 400);
    }

    $query = [
        'occurredAt' => [
            '$gte' => $startDate,
            '$lte' => $endDate
        ]
    ];

    foreach ($filters as $key => $value) {
        $query[$key] = $value;
    }

    $records = $this->mongoDB->exception_records->find($query);

    return iterator_to_array($records);
}

/**
 * 标记异常事件为已处理。
 *
 * @param string $eventId 异常事件的唯一标识
 * @return void
 * @throws Exception 如果事件不存在或已处理，抛出异常
 */
public function MarkEventAsProcessed(string $eventId): void
{
    $result = $this->mongoDB->exception_records->updateOne(
        ['_id' => new MongoDB\BSON\ObjectID($eventId), 'status' => 'unresolved'],
        ['$set' => ['status' => 'resolved']]
    );

    if ($result->getModifiedCount() === 0) {
        throw new Exception(' 事件不存在或已处理', 404);
    }
}

```

```

}

/**
 * 删除异常事件记录。
 *
 * @param string $eventId 异常事件的唯一标识
 * @return void
 * @throws Exception 如果事件不存在，抛出异常
 */
public function DeleteEvent(string $eventId): void
{
    $result = $this->mongoDB->exception_records->deleteOne(['_id' => new
MongoDB\BSON\ObjectId($eventId)]);

    if ($result->getDeletedCount() === 0) {
        throw new Exception('事件不存在', 404);
    }
}

/**
 * 生成异常事件的报告。
 *
 * @param DateTime $startDate 开始时间
 * @param DateTime $endDate 结束时间
 * @param array $filters 筛选条件
 * @return string 返回报告的内容
 * @throws Exception 如果日期范围不合法，抛出异常
 */
public function GenerateReport(DateTime $startDate, DateTime $endDate, array
$filters = []): string
{
    $events = $this->RetrieveExceptionEvents($startDate, $endDate, $filters);

    $reportContent = "异常事件报告:\n\n";
    foreach ($events as $event) {
        $reportContent .= "时间: {$event['occurredAt']}, 类型:
{$event['eventType']}, 描述: {$event['description']}, 状态: {$event['status']}\n";
    }

    return $reportContent;
}

}

// GraphQL Service provider
Route::group(['prefix' => 'graphql'], function () {
    Route::post('/', function (Request $request) {
        // 处理GraphQL调用并调用的ExceptionEvent方法
    });
});

```

// 需要更多方法和更复杂的逻辑针对GraphQLrequest体等

```
<namespace net.vnmmp.gqhad;
?php
```

// 消息体接口访问日志管理类，采用CQRS和Actor模型结构

```
class ApiAccessLogManager
```

```
{
    private $db;

    public function __construct($dbConnection)
    {
        $this->db = $dbConnection;
    }

    /**
     * 创建访问记录
     *
     * @param string $apiName 接口名称
     * @param string $apiPath 接口路径
     * @param string $callerSystem 调用方系统
     * @param DateTime $requestTime 请求时间
     * @param DateTime $responseTime 响应时间
     * @param int $durationMs 处理耗时
     * @param string|null $requestParams 请求参数
     * @param string $responseCode 响应状态码
     * @param int|null $dataCount 返回数据
     * @param string|null $errorMessage 错误信息
     *
     * @throws Exception 当数据库操作失败
     */
    public function createLog(
        string $apiName,
        string $apiPath,
        string $callerSystem,
        DateTime $requestTime,
        DateTime $responseTime,
        int $durationMs,
        ?string $requestParams,
        string $responseCode,
        ?int $dataCount,
        ?string $errorMessage
    ): void {
        // 使用工作单元模式实现实体实例化
        $this->db->beginTransaction();
        try {
            $insertQuery = "INSERT INTO tb_api_access
                (api_name, api_path, caller_system, request_time,
response_time,
                duration_ms, request_params, response_code, data_count,
error_message)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
```

```

$stmt = $this->db->prepare($insertQuery);
$stmt->execute([
    $apiName,
    $apiPath,
    $callerSystem,
    $requestTime->format('Y-m-d H:i:s'),
    $responseTime->format('Y-m-d H:i:s'),
    $durationMs,
    $requestParams,
    $responseCode,
    $dataCount,
    $errorMessage,
]);

$this->db->commit();
} catch (Exception $e) {
    $this->db->rollBack();
    throw new Exception("Log creation failed: " . $e->getMessage(), 0, $e);
}
}

/**
 * 更新访问记录
 *
 * @param int $id 访问记录ID
 * @param string|null $apiName 接口名称
 * @param string|null $apiPath 接口路径
 * @param string|null $callerSystem 调用方系统
 * @param DateTime|null $requestTime 请求时间
 * @param DateTime|null $responseTime 响应时间
 * @param int|null $durationMs 处理耗时
 * @param string|null $requestParams 请求参数
 * @param string|null $responseCode 响应状态码
 * @param int|null $dataCount 返回数据
 * @param string|null $errorMessage 错误信息
 *
 * @throws Exception 当数据库操作失败
 */
public function updateLog(
    int $id,
    ?string $apiName,
    ?string $apiPath,
    ?string $callerSystem,
    ?DateTime $requestTime,
    ?DateTime $responseTime,
    ?int $durationMs,
    ?string $requestParams,
    ?string $responseCode,
    ?int $dataCount,
    ?string $errorMessage
): void {
    // 工作单元模式应用于实体应用变更
    $this->db->beginTransaction();

```

```

try {
    $updateFields = [];
    $parameters = [];
    if ($apiName != null) {
        $updateFields[] = "api_name = ?";
        $parameters[] = $apiName;
    }
    if ($apiPath != null) {
        $updateFields[] = "api_path = ?";
        $parameters[] = $apiPath;
    }

    if ($callerSystem != null) {
        $updateFields[] = "caller_system = ?";
        $parameters[] = $callerSystem;
    }
    if ($requestTime != null) {
        $updateFields[] = "request_time = ?";
        $parameters[] = $requestTime->format('Y-m-d H:i:s');
    }
    if ($responseTime != null) {
        $updateFields[] = "response_time = ?";
        $parameters[] = $responseTime->format('Y-m-d H:i:s');
    }
    if ($durationMs != null) {
        $updateFields[] = "duration_ms = ?";
        $parameters[] = $durationMs;
    }
    if ($requestParams != null) {
        $updateFields[] = "request_params = ?";
        $parameters[] = $requestParams;
    }
    if ($responseCode != null) {
        $updateFields[] = "response_code = ?";
        $parameters[] = $responseCode;
    }

    if ($dataCount != null) {
        $updateFields[] = "data_count = ?";
        $parameters[] = $dataCount;
    }
    if ($errorMessage != null) {
        $updateFields[] = "error_message = ?";
        $parameters[] = $errorMessage;
    }

    if (empty($updateFields)) {
        throw new Exception("No fields to update.");
    }

    $parameters[] = $id;
    // 为ID添加最后的参数
    $updateQuery = "UPDATE tb_api_access SET " . implode(", ", $updateFields) .

```

```

" WHERE id = ?";
    $stmt = $this->db->prepare($updateQuery);
    $stmt->execute($parameters);

    $this->db->commit();
} catch (Exception $e) {
    $this->db->rollBack();
    throw new Exception("Log update failed: " . $e->getMessage(), 0, $e);
}
}

/**
 * 删除访问记录
 *
 * @param int $id 访问记录ID
 *
 * @throws Exception 当数据库操作失败
 */
public function deleteLog(int $id): void
{
    // 使用工作单元模式实施实体清理
    $this->db->beginTransaction();
    try {
        $deleteQuery = "DELETE FROM tb_api_access WHERE id = ?";
        $stmt = $this->db->prepare($deleteQuery);
        $stmt->execute([$id]);

        $this->db->commit();
    } catch (Exception $e) {
        $this->db->rollBack();
        throw new Exception("Log deletion failed: " . $e->getMessage(), 0, $e);
    }
}

/**
 * 根据ID查询访问记录
 *
 * @param int $id 访问记录ID
 * @return array|null 访问记录数据或null
 *
 * @throws Exception 当数据库操作失败
 */
public function getLogById(int $id): ?array
{
    try {
        $selectQuery = "SELECT * FROM tb_api_access WHERE id = ?";
        $stmt = $this->db->prepare($selectQuery);
        $stmt->execute([$id]);

        return $stmt->fetch(PDO::FETCH_ASSOC) ?: null;
    } catch (Exception $e) {
        throw new Exception("Unable to fetch log: " . $e->getMessage(), 0, $e);
    }
}

```

```

    }
}

/**
 * 分页查询访问记录
 *
 * @param int $limit 分页
 * @param int|null $afterId 游标 ID, 通常是上一页最后记录的 ID
 * @return array 访问记录数据
 *
 * @throws Exception 当数据库操作失败
 */
public function getLogs(int $limit, ?int $afterId = null): array
{
    try {
        $selectQuery = "SELECT * FROM tb_api_access" . ($afterId ? " WHERE id > ?"
: "") . " ORDER BY id ASC LIMIT ?";
        $stmt = $this->db->prepare($selectQuery);
        $params = $afterId ? [$afterId, $limit] : [$limit];
        $stmt->execute($params);

        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (Exception $e) {
        throw new Exception("Unable to fetch logs: " . $e->getMessage(), 0, $e);
    }
}

?>
<?php

class AccessLog
{
    private $entityManager;

    public function __construct($entityManager)
    {
        $this->entityManager = $entityManager;
    }

    /**
     * Spawn a new access log entry.
     *
     * This function records a new access event for a given user. The event is
     * persisted to the database for future reference.
     *
     * @param string $userId The unique identifier of the user accessing the interface.
     * @param string $status The status of the access (e.g., success, failure).
     * @param string $endpoint The API endpoint that was accessed.
     * @return void
     */
    public function spawnAccessLog(string $userId, string $status, string $endpoint):
void

```

```

{
    $accessLogEvent = new AccessLogEvent($userId, $status, $endpoint);
    $this->entityManager->persist($accessLogEvent);
    $this->entityManager->flush();
}

/**
 * Fetch access logs based on filters.
 *
 * This function retrieves access logs for a defined period and allows
 * filtering by status and user identity. The results are returned as an
 * array of access log events.
 *
 * @param DateTime $startDate The start date for filtering access logs.
 * @param DateTime $endDate The end date for filtering access logs.
 * @param string|null $status Optional status filter for logs.
 * @param string|null $userId Optional user ID filter for logs.
 * @return AccessLogEvent[] Returns an array of filtered access log events.
 */
public function fetchAccessLogs(DateTime $startDate, DateTime $endDate, ?string
$status = null, ?string $userId = null): array
{
    $queryBuilder = $this->entityManager->createQueryBuilder();
    $queryBuilder->select('log')
        ->from('AccessLogEvent', 'log')
        ->where('log.timestamp BETWEEN :startDate AND :endDate')
        ->setParameter('startDate', $startDate)
        ->setParameter('endDate', $endDate);

    if ($status !== null) {
        $queryBuilder->andWhere('log.status = :status')
            ->setParameter('status', $status);
    }

    if ($userId !== null) {
        $queryBuilder->andWhere('log.userId = :userId')
            ->setParameter('userId', $userId);
    }

    return $queryBuilder->getQuery()->getResult();
}

/**
 * Generate access report for a specific time frame.
 *
 * This function compiles a report of access logs, summarizing the volume of
 * access, frequency of endpoint calls, and any abnormal access attempts.
 *
 * @param DateTime $startDate The starting date for the report.
 * @param DateTime $endDate The ending date for the report.
 * @return array Returns a structured report containing access statistics.
 */

```

```

public function generateAccessReport(DateTime $startDate, DateTime $endDate): array
{
    $logs = $this->fetchAccessLogs($startDate, $endDate);
    $report = [
        'totalAccesses' => count($logs),
        'successfulAccesses' => 0,
        'failedAccesses' => 0,
        'endpoints' => []
    ];

    foreach ($logs as $log) {
        if ($log->getStatus() === 'success') {
            $report['successfulAccesses']++;
        } else {
            $report['failedAccesses']++;
        }

        if (!array_key_exists($log->getEndpoint(), $report['endpoints'])) {
            $report['endpoints'][$log->getEndpoint()] = 0;
        }
        $report['endpoints'][$log->getEndpoint()]++;
    }

    return $report;
}

/**
 * Export generated reports to a CSV format.
 *
 * This function accepts a generated report array and converts it into a
 * downloadable CSV file. The data will be written directly to the output stream.
 *
 * @param array $report The report data to be exported.
 * @return void
 */
public function exportReportToCsv(array $report): void
{
    header('Content-Type: text/csv');
    header('Content-Disposition: attachment;filename="access_report.csv"');

    $output = fopen('php://output', 'w');
    fputcsv($output, ['Endpoint', 'Access Count']);

    foreach ($report['endpoints'] as $endpoint => $count) {
        fputcsv($output, [$endpoint, $count]);
    }

    fclose($output);
    exit();
}

```

```

}

<?php

// record传输模型，用于传递营销费用分析的相关record
class CostAnalysisDTO {
    public function __construct(
        public ?int $id = null,
        public string $costType,
        public ?int $campaignId = null,
        public ?string $campaignName = null,
        public ?int $productId = null,
        public ?string $productName = null,
        public float $costAmount,
        public string $costDate,
        public string $channel,
        public ?int $impressionCount = null,
        public ?int $clickCount = null,
        public ?int $conversionCount = null,
        public ?float $roi = null,
        public ?DateTime $createTime = null
    ) {}
}

// 接口定义，提供营销费用分析功能
interface ICostAnalysisService {
    public function addCostAnalysis(CostAnalysisDTO $costAnalysis): string;
    public function deleteCostAnalysis(int $id): bool;
    public function editCostAnalysis(CostAnalysisDTO $costAnalysis): bool;
    public function getCostAnalysis(int $id): ?CostAnalysisDTO;
    public function queryCostAnalysis(array $criteria): array;
}

// 业务逻辑处理类，负责处理与成本分析相关的业务逻辑
class CostAnalysisService implements ICostAnalysisService {
    private $costsCollection;
    // 存储成本分析record的集合

    public function __construct($costsCollection) {
        $this->costsCollection = $costsCollection;
    }
    // 设置record集合
}

/**
 * 添加新的成本分析记录
 *
 * @param CostAnalysisDTO $costAnalysis - 成本分析对象
 * @return string - 返回记录的ID
 * @throws Exception - 抛出异常用于捕获失败情形
 */
public function addCostAnalysis(CostAnalysisDTO $costAnalysis): string {
    Assert::notNull($costAnalysis->costType, 'Cost type cannot be null');
}

```

```
Assert::notNull($costAnalysis->costAmount, 'Cost amount cannot be null');
Assert::notNull($costAnalysis->costDate, 'Cost date cannot be null');
Assert::notNull($costAnalysis->channel, 'Channel cannot be null');
```

```
$costAnalysisData = [
    'costType' => $costAnalysis->costType,
    'campaignId' => $costAnalysis->campaignId,
    'campaignName' => $costAnalysis->campaignName,
    'productId' => $costAnalysis->productId,
    'productName' => $costAnalysis->productName,
    'costAmount' => $costAnalysis->costAmount,
    'costDate' => new DateTime($costAnalysis->costDate),
    'channel' => $costAnalysis->channel,
    'impressionCount' => $costAnalysis->impressionCount,
    'clickCount' => $costAnalysis->clickCount,
    'conversionCount' => $costAnalysis->conversionCount,
    'roi' => $costAnalysis->roi,
    'createTime' => new DateTime()
];
```

```
$insertResult = $this->costsCollection->insertOne($costAnalysisData);
```

```
return (string) $insertResult->getInsertedId();
// 返回新插入record的ID
```

```
}
```

```
/**
```

```
 * 删除指定的成本分析记录
 *
 * @param int $id - 记录ID
 * @return bool - 返回操作是否成功
 * @throws Exception - 抛出异常用于处理错误
 */
```

```
public function deleteCostAnalysis(int $id): bool {
    Assert::greaterThan($id, 0, 'ID must be greater than 0');
```

```

    $deleteResult = $this->costsCollection->deleteOne(['_id' => new
MongoDB\BSON\ObjectId($id)]);
```

```
    return $deleteResult->getDeletedCount() > 0;
```

```
    // 返回是否removesuccess
```

```
}
```

```
/**
```

```
 * 编辑现有的成本分析记录
 *
 * @param CostAnalysisDTO $costAnalysis - 包含更新数据的成本分析对象
 * @return bool - 返回操作是否成功
 * @throws Exception - 抛出异常用于处理错误
 */
```

```

public function editCostAnalysis(CostAnalysisDTO $costAnalysis): bool {
    Assert::notNull($costAnalysis->id, 'ID cannot be null for editing');

    $updateData = [
        '$set' => [
            'costType' => $costAnalysis->costType,
            'campaignId' => $costAnalysis->campaignId,
            'campaignName' => $costAnalysis->campaignName,
            'productId' => $costAnalysis->productId,
            'productName' => $costAnalysis->productName,
            'costAmount' => $costAnalysis->costAmount,
            'costDate' => new DateTime($costAnalysis->costDate),
            'channel' => $costAnalysis->channel,
            'impressionCount' => $costAnalysis->impressionCount,
            'clickCount' => $costAnalysis->clickCount,
            'conversionCount' => $costAnalysis->conversionCount,
            'roi' => $costAnalysis->roi,
            'createTime' => $costAnalysis->createTime ?? new DateTime()
        ]
    ];

    $updateResult = $this->costsCollection->updateOne(['_id' => new
MongoDB\BSON\ObjectId($costAnalysis->id)], $updateData);

    return $updateResult->getModifiedCount() > 0;
// 返回是否调整done
}

/**
 * 获取指定ID的成本分析记录
 *
 * @param int $id - 记录ID
 * @return CostAnalysisDTO|null - 返回成本分析对象，或null如果未找到
 * @throws Exception - 抛出异常用于处理错误
 */
public function getCostAnalysis(int $id): ?CostAnalysisDTO {
    Assert::greaterThan($id, 0, 'ID must be greater than 0');

    $costData = $this->costsCollection->findOne(['_id' => new
MongoDB\BSON\ObjectId($id)]);

    if ($costData === null) {
        return null;
// 没有找到写入返回null
    }

    return new CostAnalysisDTO(
        (int)$costData->_id,
        $costData->costType,

```

```

        $costData->campaignId,
        $costData->campaignName,
        $costData->productId,
        $costData->productName,
        (float)$costData->costAmount,
        $costData->costDate->toDateTime()->format('Y-m-d'),
        $costData->channel,
        $costData->impressionCount,
        $costData->clickCount,
        $costData->conversionCount,
        $costData->roi,
        $costData->createTime->toDateTime()
    );
}

/**
 * 根据条件查询一系列成本分析记录
 *
 * @param array $criteria - 查询条件
 * @return array - 返回符合条件的成本分析记录数组
 * @throws Exception - 抛出异常用于处理错误
 */
public function queryCostAnalysis(array $criteria): array {
    $query = [];

    if (!empty($criteria['costType'])) {
        $query['costType'] = $criteria['costType'];
    }
    if (!empty($criteria['campaignId'])) {
        $query['campaignId'] = $criteria['campaignId'];
    }
    // 其他条件可以进一步添加
    $result = $this->costsCollection->find($query);

    $costList = [];
    foreach ($result as $costData) {
        $costList[] = new CostAnalysisDTO(
            (int)$costData->_id,
            $costData->costType,
            $costData->campaignId,
            $costData->campaignName,
            $costData->productId,
            $costData->productName,
            (float)$costData->costAmount,
            $costData->costDate->toDateTime()->format('Y-m-d'),
            $costData->channel,
            $costData->impressionCount,
            $costData->clickCount,
            $costData->conversionCount,
            $costData->roi,
            $costData->createTime->toDateTime()
        );
    }
}

```

```

    }

    return $costList;
    // 返回查询到的打印日志数组
}

}

// 主要功能：新增服务和执接口
$costsCollection = new MongoDB\Collection('test', 'tb_cost_analysis');
    // 集合准备
$costAnalysisService = new CostAnalysisService($costsCollection);

// : 要添加的log
$analysisDTO = new CostAnalysisDTO(
    null,
    '广告费用',
    null,
    '黑色星期五促销',
    null,
    null,
    10000.00,
    '2023-11-01',
    '在线',
    100000,
    5000,
    300,
    0.10
);

// 添加打印日志
$newId = $costAnalysisService->addCostAnalysis($analysisDTO);
echo "新添加记录的ID为: $newId\n";

// 查询log
$costRecord = $costAnalysisService->getCostAnalysis((int)$newId);
echo "查询到的成本记录: ";
print_r($costRecord);
<?php

```

```
namespace App\Http\Controllers;
```

```
use App\Services\MarketingExpenseService;
use Illuminate\Http\Request;
```

```
/**
 * Class MarketingExpenseController
 *
 * 处理营销费用的相关请求，包括获取费用报告、按维度分析及导出报表等功能。
 *
 * 该控制器提供了灵活的API接口，方便用户进营销费用的管理和分析。所有方法都经过保护，确

```

保请求的安全性和有效性。

```

    */
class MarketingExpenseController extends Controller
{
    protected $expenseService;

    /**
     * MarketingExpenseController constructor.
     *
     * @param MarketingExpenseService $expenseService 注入营销费用服务，遵循依赖注入原则，利用服务容器提供服务实例。
     */
    public function __construct(MarketingExpenseService $expenseService)
    {
        $this->expenseService = $expenseService;
    }

    /**
     * 获取营销费用报告。
     *
     * @param Request $request 包含查询参数，如起始和结束时间、活动渠道、分页信息等。
     *
     * @return \Illuminate\Http\JsonResponse 返回格式化的费用报告数据，包括总费用、各活动费用等。
     *
     * 该方法针对用户的查询需求动态返回数据，支持分页功能，确保用户能高效地获取数据。
     */
    public function getExpenseReport(Request $request)
    {
        // 从request中get分页参数及筛选条件
        $page = $request->input('page', 1);
        // 兜底第1页
        $size = $request->input('size', 10);
        // 基础每页10条record
        $filters = $request->only(['start_date', 'end_date', 'channel', 'activity_type']);

        // 调用服务层查询费用报告
        $reportData = $this->expenseService->getExpenses($filters, $page, $size);

        return response()->json($reportData);
    }

    /**
     * 导出费用分析结果。
     *
     * @param Request $request 包含导出相关的参数，例如选择的时间范围和渠道。
     *
     * @return \Illuminate\Http\JsonResponse 返回导出操作的状态及相关信息。
     *
     * 此方法使用后台任务处理导出请求，确保API请求快速响应，同时将导出任务交由异步处理。
     */

```

```

public function exportExpenseReport(Request $request)
{
    $filters = $request->only(['start_date', 'end_date', 'channel',
'activity_type']);

    // 将导出任务dispatch到队列中，延迟处理导出
    $job = $this->expenseService->dispatchExportJob($filters);

    return response()->json(['message' => '导出任务已开始处理', 'job_id' => $job-
>getJobId()]);
}

}

namespace App\Services;

use App\Repositories\MarketingExpenseRepository;
use Illuminate\Support\Facades\Queue;

/**
 * Class MarketingExpenseService
 *
 * 提供营销费用相关的业务逻辑处理，包括费用数据获取、导出任务处理等。
 *
 * 该类封装了业务逻辑，原则上不直接与数据源交互，而是通过仓储接口访问。
 */
class MarketingExpenseService
{
    protected $expenseRepository;

    /**
     * MarketingExpenseService constructor.
     *
     * @param MarketingExpenseRepository $expenseRepository 依赖注入，封装数据访问逻辑
     *
     */
    public function __construct(MarketingExpenseRepository $expenseRepository)
    {
        $this->expenseRepository = $expenseRepository;
    }

    /**
     * 根据过滤条件获取营销费用数据。
     *
     * @param array $filters 包含筛选条件的数组，例如日期范围、渠道、活动类型。
     * @param int $page 当前请求的页码。
     * @param int $size 每页显示的记录数。
     *
     * @return array 返回包含费用数据的数组和分页信息。
     *
     * 此函数确保可以动态按条件检索费用数据，支持偏移分页实现。
     */

```

```

public function getExpenses(array $filters, int $page = 1, int $size = 10): array
{
    // 计算偏移
    $offset = ($page - 1) * $size;

    // 调用仓储方法get信息
    $expenses = $this->expenseRepository->fetchExpenses($filters, $offset, $size);

    // 在这里可以添加更多的业务逻辑，比如写入处理或格式化

    return [
        'total' => $this->expenseRepository->countExpenses($filters),
// 总打印日志数
        'items' => $expenses,
        // 本次input的有效载荷项
        'page' => $page,
        'size' => $size,
    ];
}

/**
 * 将导出任务添加到队列。
 *
 * @param array $filters 包含导出条件的参数，例如日期范围，渠道等。
 *
 * @return \Illuminate\Foundation\Bus\PendingDispatch 返回待处理的队列任务实例。
 *
 * 该方法使用Laravel队列系统优化性能，允许用户下一次操作，长时间等待。
 */
public function dispatchExportJob(array $filters)
{
    return Queue::push(new \App\Jobs\ExportMarketingExpenseJob($filters));
}
}

namespace App\Repositories;

use App\Models\MarketingExpense;

/**
 * Class MarketingExpenseRepository
 *
 * 封装营销费用相关的数据访问逻辑，负责与数据库交互。
 *
 * 此类通过Eloquent ORM提供了对营销费用数据的统一访问接口，于域模型的抽象。
 */
class MarketingExpenseRepository
{
    /**
     * 根据过滤条件获取营销费用数据。
     *
     * @param array $filters 过滤条件，比如开始时间、结束时间等。

```

```

* @param int $offset 偏移，支持分页。
* @param int $limit 获取的记录数。
*
* @return \Illuminate\Database\Eloquent\Collection 返回符合条件的营销费用数据集合
。
*
* 此方法确保了数据访问逻辑的分离，支持灵活的查询。
*/
public function fetchExpenses(array $filters, int $offset, int $limit)
{
    $query = MarketingExpense::query();

    // 应用过滤条件
    if (!empty($filters['start_date'])) {
        $query->where('date', '=', $filters['start_date']);
    }
    if (!empty($filters['end_date'])) {
        $query->where('date', '<=', $filters['end_date']);
    }
    if (!empty($filters['channel'])) {
        $query->where('channel', $filters['channel']);
    }
    if (!empty($filters['activity_type'])) {
        $query->where('activity_type', $filters['activity_type']);
    }

    return $query->offset($offset)->limit($limit)->get();
}

/**
* 计算符合条件的营销费用记录。
*
* @param array $filters 过滤条件。
*
* @return int 返回符合条件的记录，便于进分页等操作。
*
* 此方法确保了统计逻辑与查询条件的一致性，有助于向用户提供准确的数据视图。
*/
public function countExpenses(array $filters): int
{
    $query = MarketingExpense::query();

    if (!empty($filters['start_date'])) {
        $query->where('date', '=', $filters['start_date']);
    }
    if (!empty($filters['end_date'])) {
        $query->where('date', '<=', $filters['end_date']);
    }
    if (!empty($filters['channel'])) {
        $query->where('channel', $filters['channel']);
    }
    if (!empty($filters['activity_type'])) {

```

```

        $query->where('activity_type', $filters['activity_type']);
    }

    return $query->count();
}

}

namespace App\Jobs;

use App\Services\MarketingExpenseService;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

/**
 * Class ExportMarketingExpenseJob
 *
 * 处理营销费用的导出任务。
 *
 * 该类实现了队列处理接口，以便在后台异步处理导出请求，占用API请求的响应时间。
 */
class ExportMarketingExpenseJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $filters;

    /**
     * ExportMarketingExpenseJob constructor.
     *
     * @param array $filters 包含导出所需的参数，例如开始时间和渠道。
     */
    public function __construct(array $filters)
    {
        $this->filters = $filters;
    }

    /**
     * 执导出任务。
     *
     * @return void
     *
     * 该方法封装了具体的导出逻辑，例如生成CSV文件，将文件存储到本地或云端等。
     */
    public function handle()
    {
        // 执具体的导出逻辑，例如生成CSV或JSON等格式
        // 将抓取到的payload写入文件并存储到指定位置
    }
}

```

```

        // 在这里添加文件处理和存储逻辑
    }
}

<?php

namespace App\Http\Controllers\API\V1;

use App\Http\Controllers\Controller;
use App\Models\DataQuality;
use Illuminate\Http\Request;

class DataQualityController extends Controller
{
    /**
     * 添加新的数据质监控记录
     *
     * @param Request $request 包含数据质监控记录的所有属性
     * @return \Illuminate\Http\JsonResponse 返回新增记录的详细信息
     */
    public function createRecord(Request $request)
    {
        $dataQuality = new DataQuality();
        $dataQuality->source_id = $request->input('source_id');
        $dataQuality->source_name = $request->input('source_name');
        $dataQuality->check_type = $request->input('check_type');
        $dataQuality->metrics_name = $request->input('metrics_name');
        $dataQuality->metrics_value = $request->input('metrics_value');
        $dataQuality->threshold = $request->input('threshold');
        $dataQuality->status = $request->input('status');
        $dataQuality->check_time = $request->input('check_time');
        $dataQuality->duration_seconds = $request->input('duration_seconds');
        $dataQuality->error_count = $request->input('error_count');
        $dataQuality->sample_data = $request->input('sample_data');
        $dataQuality->save();

        return response()->json($dataQuality, 201);
    }

    /**
     * 获取所有数据质监控记录
     *
     * @return \Illuminate\Http\JsonResponse 返回所有记录的数组
     */
    public function getAllRecords()
    {
        $dataQualities = DataQuality::all();
        return response()->json($dataQualities);
    }
}

```

```

/**
 * 获取特定数据质监控记录的详细信息
 *
 * @param int $id 监控记录的ID
 * @return \Illuminate\Http\JsonResponse 返回该记录的详细信息
 */
public function getRecordById($id)
{
    $dataQuality = DataQuality::find($id);
    if (!$dataQuality) {
        return response()->json(['message' => '记录未找到'], 404);
    }
    return response()->json($dataQuality);
}

/**
 * 更新现有的数据质监控记录
 *
 * @param Request $request 包含更新后的记录属性
 * @param int $id 监控记录的ID
 * @return \Illuminate\Http\JsonResponse 返回更新后的记录信息
 */
public function updateRecord(Request $request, $id)
{
    $dataQuality = DataQuality::find($id);
    if (!$dataQuality) {
        return response()->json(['message' => '记录未找到'], 404);
    }

    $dataQuality->source_id = $request->input('source_id', $dataQuality->source_id);
    $dataQuality->source_name = $request->input('source_name', $dataQuality->source_name);
    $dataQuality->check_type = $request->input('check_type', $dataQuality->check_type);
    $dataQuality->metrics_name = $request->input('metrics_name', $dataQuality->metrics_name);
    $dataQuality->metrics_value = $request->input('metrics_value', $dataQuality->metrics_value);
    $dataQuality->threshold = $request->input('threshold', $dataQuality->threshold);
    $dataQuality->status = $request->input('status', $dataQuality->status);
    $dataQuality->check_time = $request->input('check_time', $dataQuality->check_time);
    $dataQuality->duration_seconds = $request->input('duration_seconds', $dataQuality->duration_seconds);
    $dataQuality->error_count = $request->input('error_count', $dataQuality->error_count);
    $dataQuality->sample_data = $request->input('sample_data', $dataQuality->sample_data);
    $dataQuality->save();
}

```

```

        return response()->json($dataQuality);
    }

/**
 * 删除指定的数据质监控记录
 *
 * @param int $id 监控记录的ID
 * @return \Illuminate\Http\JsonResponse 返回删除结果的消息
 */
public function deleteRecord($id)
{
    $dataQuality = DataQuality::find($id);
    if (!$dataQuality) {
        return response()->json(['message' => '记录未找到'], 404);
    }

    $dataQuality->delete();
    return response()->json(['message' => '记录已删除']);
}

/**
 * 根据条件查询数据质监控结果
 *
 * @param Request $request 查询条件
 * @return \Illuminate\Http\JsonResponse 返回条件的记录数组
 */
public function queryRecords(Request $request)
{
    $query = DataQuality::query();

    if ($request->has('source_id')) {
        $query->where('source_id', $request->input('source_id'));
    }

    if ($request->has('status')) {
        $query->where('status', $request->input('status'));
    }

    if ($request->has('check_time')) {
        $query->whereDate('check_time', $request->input('check_time'));
    }

    $dataQualities = $query->get();
    return response()->json($dataQualities);
}
}
<?php

/**
 * Class DataQualityMonitor

```

```

*
* 该类用于实现电商平台的数据质监控与管理功能。
* 提供获取监控结果、生成报告、处理异常数据等功能。
*/
class DataQualityMonitor
{
    private $dbConnection;

    /**
     * 数据质监控构造函数
     *
     * @param PDO $dbConnection 数据库连接对象
     */
    public function __construct(PDO $dbConnection)
    {
        $this->dbConnection = $dbConnection;
    }

    /**
     * 获取数据质监控的结果
     *
     * @return array 返回监控结果数组
     * @throws Exception 数据库查询失败时抛出异常
     */
    public function getMonitoringResults(): array
    {
        // 1. 执查询find监控结果
        $query = "SELECT * FROM data_quality_monitoring";
        $statement = $this->dbConnection->query($query);

        // 2. 检查查询结果
        if (!$statement) {
            throw new Exception('Unable to retrieve monitoring results.');
        }

        // 3. 抓取结果并将其解析为数组
        $results = $statement->fetchAll(PDO::FETCH_ASSOC);

        // 4. 返回监控结果
        return $results;
    }

    /**
     * 生成数据质监控报告
     *
     * @param array $monitoringResults 监控结果数组
     * @return string 返回生成的报告
     */
    public function generateReport(array $monitoringResults): string
    {
        // 1. 检查监控结果是否为空

```

```

    if (empty($monitoringResults)) {
        return "No monitoring results available.";
    }

    // 2. bootstrap报告内容
    $reportContent = "Data Quality Monitoring Report\n\n";

    // 3. 按构建报告内容
    foreach ($monitoringResults as $result) {
        $reportContent .= sprintf(
            "Data ID: %s, Status: %s, Quality Score: %s\n",
            $result['data_id'],
            $result['status'],
            $result['quality_score']
        );
    }

    // 4. 返回生成的报告内容
    return $reportContent;
}

/**
 * 处理异常数据
 *
 * @param array $exceptionData 异常数据数组
 * @return bool 返回处理状态
 * @throws Exception 处理失败时抛出异常
 */
public function handleExceptionData(array $exceptionData): bool
{
    // 先得到互斥锁来确保线程安全
    $lock = new Mutex();

    // 1. 加载锁
    $lock->lock();

    try {
        // 2. 筛选异常payload
        foreach ($exceptionData as $data) {
            $this->archiveData($data['data_id']);
            $this->notifyRelevantTeams($data['data_id']);
        }

        // 3. 表示OK处理
        return true;
    } catch (Exception $e) {
        throw new Exception("Failed to handle exception data: " . $e-
>getMessage());
    } finally {
        // 4. 释放锁

```

```

        $lock->unlock();
    }

}

/**
 * 将异常数据归档
 *
 * @param string $dataId 待归档的数据ID
 * @return void
 * @throws Exception 归档失败时抛出异常
 */
private function archiveData(string $dataId): void
{
    // 1. 准备归档SQL
    $archiveQuery = "INSERT INTO archived_data (data_id) VALUES (:data_id)";
    $statement = $this->dbConnection->prepare($archiveQuery);

    // 2. 执归档操作
    if (!$statement->execute([':data_id' => $dataId])) {
        throw new Exception('Failed to archive data.');
```

```

        // 广播搞定
    }
}

// 使用（应放在其他文件中用来实例化类）
try {
    $pdo = new PDO('mysql:host=192.168.0.1;dbname=testdb', 'username', 'password');
    $monitor = new DataQualityMonitor($pdo);

    $results = $monitor->getMonitoringResults();
    $report = $monitor->generateReport($results);

    // 获得的异常有效载荷
    $exceptionData = [['data_id' => '123'], ['data_id' => '456']];
    $monitor->handleExceptionData($exceptionData);

    echo $report;
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}

<?php

```

```

namespace App\Repository;

```

```

use App\Model\DataSource;
use PDO;

```

```

/**
 * Class DataSourceRepository
 * 该类负责与 `tb_data_source` 表进行数据交互，封装了所有对数据源的CRUD操作。
 * 采用仓储模式，确保业务逻辑与数据访问的完全解耦。
 */
class DataSourceRepository
{
    private $pdo;

    public function __construct(PDO $pdo)
    {
        $this->pdo = $pdo;
    }

    /**
     * Spawn a new Data Source.
     *
     * @param DataSource $dataSource - 数据源模型，包括所有必要配置。
     * @return string - 返回生成的数据源ID（GUID）。
     *
     * 这个函数负责在 `tb_data_source` 表中创建新的记录。我们使用事务确保数据一致性。
     * 在未来的实现中，需要确保连接配置的有效性，并加入更详细的异常处理。
     */
}

```

```

{
    // , 实现取决于有效载荷访问层
    return null;
// 未找到
}

/**
 * 删除任务配置。
 *
 * @param string $taskId - 待删除的任务配置唯一标识符。
 *
 * @throws \InvalidArgumentException - 当任务ID无效时抛出异常。
 *
 * @return void
 */
public function deleteTask(string $taskId): void
{
    $existingTaskConfig = $this->loadFromDatabase($taskId);

    if ($existingTaskConfig === null) {
        throw new \InvalidArgumentException("无效的任务ID, 无法删除。");
    }

    // 从payload源中废弃任务选项
    $this->removeFromDatabase($taskId);
}

/**
 * 从数据库中移除任务配置。
 *
 * @param string $taskId - 待删除的任务配置唯一标识符。
 *
 * @return void
 */
private function removeFromDatabase(string $taskId): void
{
    // 干掉操作, 逻辑封装在此函数中
}

/**
 * 浏览现有任务模板。
 *
 * 用户可以查看所有可用的任务模板, 便于选择合适的基础配置。
 *
 * @return array - 返回当前所有的任务模板。
 */
public function browseTemplates(): array
{
    return $this->taskTemplates;
}
}

```

<?php

```

declare(strict_types=1);

namespace RecommendationManagement;

use PDO;

class RecommendationController
{
    private RecommendationService $recommendationService;

    public function __construct(RecommendationService $recommendationService)
    {
        $this->recommendationService = $recommendationService;
    }

    /**
     * Spawn a new recommendation record.
     *
     * @param array $data An associative array containing recommendation details.
     * @return void
     * @throws \Exception if the spawn process encounters an issue.
     */
    public function SpawnRecommendation(array $data): void
    {
        $this->recommendationService->CreateRecommendation($data);
    }

    /**
     * Delete an existing recommendation by its ID.
     *
     * @param int $id The recommendation ID to be deleted.
     * @return void
     * @throws \Exception if the delete operation fails.
     */
    public function DeleteRecommendation(int $id): void
    {
        $this->recommendationService->RemoveRecommendation($id);
    }

    /**
     * Edit recommendation details.
     *
     * @param int $id The recommendation ID to be edited.
     * @param array $data The new recommendation data.
     * @return void
     * @throws \Exception if the update fails due to version mismatch.
     */
    public function EditRecommendation(int $id, array $data): void
    {
        $this->recommendationService->UpdateRecommendation($id, $data);
    }
}

```

```

/**
 * Retrieve details of a specific recommendation.
 *
 * @param int $id The recommendation ID to be retrieved.
 * @return array The recommendation details.
 * @throws \Exception if the recommendation cannot be found.
 */
public function ViewRecommendationDetails(int $id): array
{
    return $this->recommendationService->FetchRecommendationDetails($id);
}

/**
 * Generate a report based on recommendations within given parameters.
 *
 * @param array $filters Set of filters for report generation.
 * @return array The generated report data.
 * @throws \Exception if the report generation fails.
 */
public function GenerateReport(array $filters): array
{
    return $this->recommendationService->ProduceReport($filters);
}

}

class RecommendationService
{
    private RecommendationRepository $recommendationRepository;

    public function __construct(RecommendationRepository $recommendationRepository)
    {
        $this->recommendationRepository = $recommendationRepository;
    }

    /**
     * Create a new recommendation in the database.
     *
     * @param array $data The recommendation data to be stored.
     * @return void
     * @throws \Exception if the database operation fails.
     */
    public function CreateRecommendation(array $data): void
    {
        $this->recommendationRepository->InsertRecommendation($data);
    }

    /**
     * Remove a recommendation from the database.
     *
     * @param int $id The recommendation ID to delete.
     * @return void
     * @throws \Exception if the recommendation cannot be found.

```

```

    */
    public function RemoveRecommendation(int $id): void
    {
        $this->recommendationRepository->DeleteRecommendationById($id);
    }

    /**
     * Update an existing recommendation with new data.
     *
     * @param int $id The ID of the recommendation to update.
     * @param array $data The new data for the recommendation.
     * @return void
     * @throws \Exception if the version check fails.
     */
    public function UpdateRecommendation(int $id, array $data): void
    {
        $this->recommendationRepository->ModifyRecommendation($id, $data);
    }

    /**
     * Fetch details of a specific recommendation.
     *
     * @param int $id The ID of the recommendation to retrieve.
     * @return array The recommendation details.
     * @throws \Exception if the recommendation does not exist.
     */
    public function FetchRecommendationDetails(int $id): array
    {
        return $this->recommendationRepository->GetRecommendationById($id);
    }

    /**
     * Produce a report based on specified filters.
     *
     * @param array $filters The filters to apply for the report.
     * @return array The report data generated from the recommendations.
     * @throws \Exception if the report generation encounters an issue.
     */
    public function ProduceReport(array $filters): array
    {
        return $this->recommendationRepository->GenerateRecommendationReport($filters);
    }
}

class RecommendationRepository
{
    private PDO $pdo;

    public function __construct(PDO $pdo)
    {
        $this->pdo = $pdo;
    }
}

```

```

}

/**
 * Insert a new recommendation record into the database.
 *
 * @param array $data The data for the new recommendation.
 * @return void
 */
public function InsertRecommendation(array $data): void
{
    $stmt = $this->pdo->prepare(
        "INSERT INTO tb_recommendation (user_id, user_name, recommended_product_id,
        recommended_product_name, algorithm_type, score, position, display_time, click_status)
        VALUES (:user_id, :user_name, :recommended_product_id,
        :recommended_product_name, :algorithm_type, :score, :position, :display_time,
        :click_status)"
    );
    $stmt->execute($data);
}

/**
 * Delete a recommendation from the database by its ID.
 *
 * @param int $id The ID of the recommendation to delete.
 * @return void
 * @throws \Exception if the recommendation does not exist.
 */
public function DeleteRecommendationById(int $id): void
{
    $stmt = $this->pdo->prepare("DELETE FROM tb_recommendation WHERE id = :id");
    $stmt->execute(['id' => $id]);
}

/**
 * Modify an existing recommendation based on ID and new data.
 *
 * @param int $id The recommendation ID to update.
 * @param array $data The new data to use for the update.
 * @return void
 * @throws \Exception if the version check fails.
 */
public function ModifyRecommendation(int $id, array $data): void
{
    // Version checking logic would be implemented here.
    $stmt = $this->pdo->prepare(
        "UPDATE tb_recommendation SET user_id = :user_id, user_name = :user_name,
        recommended_product_id = :recommended_product_id,
        recommended_product_name = :recommended_product_name, algorithm_type =
        :algorithm_type, score = :score, position = :position,
        display_time = :display_time, click_status = :click_status WHERE id = :id"
    );
    $data['id'] = $id;
    $stmt->execute($data);
}

```

```

}

/**
 * Retrieve recommendation details by its ID.
 *
 * @param int $id The recommendation ID to fetch.
 * @return array The details of the recommendation.
 * @throws \Exception if the recommendation cannot be found.
 */
public function GetRecommendationById(int $id): array
{
    $stmt = $this->pdo->prepare("SELECT * FROM tb_recommendation WHERE id = :id");
    $stmt->execute(['id' => $id]);
    return $stmt->fetch(PDO::FETCH_ASSOC);
}

/**
 * Generate a report based on provided filters.
 *
 * @param array $filters The filters for report generation.
 * @return array The resulting report data.
 * @throws \Exception if the report generation encounters an issue.
 */
public function GenerateRecommendationReport(array $filters): array
{
    // Report generation logic based on filters.
    // This is an placeholder.
    return [];
}
}
<?php

```

```

namespace App\Controller;

```

```

use App\Service\RecommendationResultService;
use App\DTO\RecommendationResultDTO;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\Routing\Annotation\Route;

```

```

/**
 * Class RecommendationResultController
 * 负责接收推荐记录相关的请求并转发到服务层
 */
class RecommendationResultController
{
    public function __construct(private RecommendationResultService
    $recommendationResultService) {}

    /**
     * @Route("/api/recommendation/results", methods={"POST"})
     */

```

```

public function addRecommendationResult(Request $request): JsonResponse
{
    $data = json_decode($request->getContent(), true);
    $resultDTO = new RecommendationResultDTO(...$data);
// 使用 DTO 封装data
    $this->recommendationResultService->addResult($resultDTO);
    return new JsonResponse(['status' => 'Result added'], 201);
}

/**
 * @Route("/api/recommendation/results", methods={"GET"})
 */
public function getRecommendationResults(Request $request): JsonResponse
{
    $page = (int) $request->query->get('page', 1);
    $size = (int) $request->query->get('size', 10);
    $results = $this->recommendationResultService->getResults($page, $size);
    return new JsonResponse($results);
}

/**
 * @Route("/api/recommendation/results/{id}", methods={"DELETE"})
 */
public function deleteRecommendationResult(int $id): JsonResponse
{
    $this->recommendationResultService->deleteResult($id);
    return new JsonResponse(['status' => 'Result deleted'], 204);
}
}

namespace App\Service;

use App\DTO\RecommendationResultDTO;
use App\Repository\IRecommendationResultRepository;

/**
 * Class RecommendationResultService
 * 核心业务逻辑服务类，负责处理推荐结果的记录，查询和管理
 */
class RecommendationResultService
{
    public function __construct(private IRecommendationResultRepository $repository) {}

    /**
     * 添加推荐结果
     * @param RecommendationResultDTO $resultDTO 推荐结果传输对象
     * @return void
     */
    public function addResult(RecommendationResultDTO $resultDTO): void

```

```

{
    // 在此添加具体的业务逻辑，例如审查打印日志是否合法
    // 若data检查通过，调用存储库方法进data持久化
    $this->repository->insert($resultDTO);
}

/**
 * 获取推荐结果列表
 * @param int $page 当前页码
 * @param int $size 每页显示的记录数
 * @return array 推荐结果的列表
 */
public function getResults(int $page, int $size): array
{
    $offset = ($page - 1) * $size;
    return $this->repository->findAll($offset, $size);
}

/**
 * 删除推荐结果
 * @param int $id 推荐结果的唯一标识符
 * @return void
 */
public function deleteResult(int $id): void
{
    $this->repository->remove($id);
}
}

namespace App\DTO;

/**
 * Class RecommendationResultDTO
 * 推荐结果数据传输对象，封装推荐结果的数据结构
 */
class RecommendationResultDTO
{
    public function __construct(
        public string $itemId,
        public string $userId,
        public float $score,
        public \DateTimeInterface $timestamp
    ) {}
}

namespace App\Repository;

use App\DTO\RecommendationResultDTO;

/**
 * Interface IRecommendationResultRepository
 * 推荐结果持久化操作接口

```

```

*/
interface IRecommendationResultRepository
{
    public function insert(RecommendationResultDTO $resultDTO): void;

    public function findAll(int $offset, int $size): array;

    public function remove(int $id): void;
}

// MongoDB 具体实现
namespace App\Repository;

use App\DTO\RecommendationResultDTO;
use MongoDB\Client;
use MongoDB\BSON\ObjectId;

/**
 * Class MongoRecommendationResultRepository
 * MongoDB 推荐结果持久化操作的具体实现
 */
class MongoRecommendationResultRepository implements IRecommendationResultRepository
{
    private $collection;

    public function __construct(Client $client)
    {
        $this->collection = $client->myDatabase->recommendation_results;
    }

    public function insert(RecommendationResultDTO $resultDTO): void
    {
        $document = [
            'itemId' => $resultDTO->itemId,
            'userId' => $resultDTO->userId,
            'score' => $resultDTO->score,
            'timestamp' => $resultDTO->timestamp
        ];
        $this->collection->insertOne($document);
    }

    public function findAll(int $offset, int $size): array
    {
        return $this->collection->find([], ['skip' => $offset, 'limit' => $size])->toArray();
    }

    public function remove(int $id): void
    {
        $this->collection->deleteOne(['_id' => new ObjectId($id)]);
    }
}

```

```

<namespace com.cn.zffjp.pazrj;
?php
// 引入MongoDB ODM库
require_once 'vendor/autoload.php';

use MongoDB\Client;

class ReportConfigManager {
    private $mongoClient;
    private $db;
    private $collection;

    public function __construct($uri, $dbName, $collectionName) {
        // initializeMongoDB连接
        $this->mongoClient = new Client($uri);
        $this->db = $this->mongoClient->$dbName;
        $this->collection = $this->db->$collectionName;
    }

    // 添加报表设置
    public function addReportConfig($data) {
        assert(!empty($data['report_name']), "报表名称不能为空");
        assert(!empty($data['report_type']), "报表类型不能为空");
        assert(!empty($data['data_source']), "数据来源不能为空");

        $data['create_time'] = new MongoDB\BSON\UTCDateTime();
        $this->collection->insertOne($data);
    }

    // 插入data
    // 清理报表属性
    public function deleteReportConfig($id) {
        assert(!empty($id), "报表配置ID不能为空");

        $result = $this->collection->deleteOne(['_id' => new
MongoDB\BSON\ObjectId($id)]);
        assert($result->getDeletedCount() > 0, "删除失败, 记录不存在");
    }

    // 校验清理结果
    // 编辑报表属性
    public function editReportConfig($id, $data) {
        assert(!empty($id), "报表配置ID不能为空");
        assert(!empty($data), "更新数据不能为空");

        $data['last_generate_time'] = new MongoDB\BSON\UTCDateTime();
        // 同步生成时间
        $result = $this->collection->updateOne(['_id' => new
MongoDB\BSON\ObjectId($id)], ['$set' => $data]);
        assert($result->getModifiedCount() > 0, "更新失败, 记录不存在或数据未变更");
    }
}

```

```

// 提取报表选项详情
public function getReportConfigDetail($id) {
    assert(!empty($id), "报表配置ID不能为空");

    $config = $this->collection->findOne(['_id' => new
MongoDB\BSON\ObjectId($id)]);
    assert($config !== null, "未找到报表配置");
    return $config;
    // 返回properties详情
}

// 查询报表
public function queryReports($filters = []) {
    $cursor = $this->collection->find($filters);
    $results = [];
    foreach ($cursor as $document) {
        $results[] = $document;
    }
    // 累加查询结果
    return $results;
}
// 返回查询结果
}

// 使用
$manager = new ReportConfigManager('mongodb:
//192.168.0.1:27017', 'report_db', 'tb_report_config');

try {
    $manager->addReportConfig([
        'report_name' => '月度销售报表',
        'report_type' => '销售',
        'data_source' => '销售数据库',
        'metrics_config' => json_encode(['总销售额', '销售']),
        'dimension_config' => json_encode(['地区', '销售员']),
        'schedule_type' => '每月',
        'status' => '激活',
    ]);

    $configs = $manager->queryReports();
    foreach ($configs as $config) {
        echo json_encode($config);
    }
} catch (Exception $e) {
    echo $e->getMessage();
    // 故障处理
}
<?php
declare(strict_types=1);

/**

```

```

* Class ReportManager
* 该类负责报表的创建、修改、管理和浏览操作。
*/
class ReportManager {
    private array $reports = [];

    /**
     * 创建新报表
     *
     * @param string $name 报表名称
     * @param string $template 模板选择
     * @param array $dataSource 数据来源
     * @return int 返回新创建报表的ID
     */
    public function createReport(string $name, string $template, array $dataSource):
    int {
        // 生成新的报表ID
        $reportId = count($this->reports) + 1;
        // 注册报表entity
        $this->reports[$reportId] = [
            'id' => $reportId,
            'name' => $name,
            'template' => $template,
            'dataSource' => $dataSource,
            'version' => 1,
        ];
        return $reportId;
    }
    // 返回新插入报表的ID
}

/**
 * 修改现有报表
 *
 * @param int $reportId 报表ID
 * @param string $name 新的报表名称
 * @param string $template 新的模板
 * @param array $dataSource 新的数据来源
 * @return bool 修改是否成功
 */
    public function modifyReport(int $reportId, string $name, string $template, array
$dataSource): bool {
        // 核实报表是否存在
        if (!isset($this->reports[$reportId])) {
            return false;
        }
        // 报表不存在，修改未完成
        // 修改报表内容
        $this->reports[$reportId]['name'] = $name;
        $this->reports[$reportId]['template'] = $template;
        $this->reports[$reportId]['dataSource'] = $dataSource;
        return true;
    }
    // 返回修改done
}

```

```

/**
 * 删除现有报表
 *
 * @param int $reportId 报表ID
 * @return bool 删除是否成功
 */
public function deleteReport(int $reportId): bool {
    // 核实报表是否存在
    if (!isset($this->reports[$reportId])) {
        return false;
        // 报表不存在，注销凉了
    }

    // 从数组中移除报表
    unset($this->reports[$reportId]);
    return true;
// 返回注销done
}

/**
 * 浏览报表模板
 *
 * @return array 返回可用的报表模板
 */
public function browseTemplates(): array {
    // 返回可用的报表模板
    return [
        '模板1' => 'Sales Report',
        '模板2' => 'Inventory Report',
        '模板3' => 'Performance Report',
    ];
}

/**
 * 查看报表详细信息
 *
 * @param int $reportId 报表ID
 * @return array|null 返回报表详细信息或null
 */
public function viewReport(int $reportId): ?array {
    // validate报表是否存在
    if (isset($this->reports[$reportId])) {
        return $this->reports[$reportId];
// 返回报表详细信息
    }

    return null;
// 报表不存在，返回null
}

/**
 * 获取所有报表

```

```

*
* @return array 返回所有报表
*/
public function listReports(): array {
    return $this->reports;
    // 返回所有报表即其array
}

/**
* 管理
*
* @param int $reportId 报表ID
* @return bool 更新是否成功
*/
public function versionControl(int $reportId): bool {
    // 证实报表是否存在
    if (!isset($this->reports[$reportId])) {
        return false;
        // 报表不存在，变更出错
    }
    // 增加号
    $this->reports[$reportId]['version'] += 1;
    return true;
    // 返回覆盖搞定
}

/**
* 筛选报表
*
* @param string $criteria 筛选条件
* @return array 返回符合条件的报表
*/
public function filterReports(string $criteria): array {
    // 装载过滤后的报表数组
    $filteredReports = [];
    // 遍历所有报表
    foreach ($this->reports as $report) {
        // 根据条件筛选报表
        if (strpos($report['name'], $criteria) !== false) {
            $filteredReports[] = $report;
        }
    }
    return $filteredReports;
    // 返回符合条件的报表数组
}

/**
* Class ReportPlugin
* 此类扩展了报表管理功能
*/
class ReportPlugin {

```

```

private ReportManager $reportManager;

public function __construct(ReportManager $reportManager) {
    $this->reportManager = $reportManager;
    // 注入ReportManager
}

/**
 * 上传报表数据
 *
 * @param int $reportId 报表ID
 * @param array $data 报表数据
 * @return bool 上传是否成功
 */
public function uploadReportData(int $reportId, array $data): bool {
    // 确保报表是否存在
    if (is_null($this->reportManager->viewReport($reportId))) {
        return false;
    }
    // 报表不存在，上传出错
    // 在此实现上传信息逻辑...
    // 信息上传搞定
    return true;
    // 返回上传启动完毕
}
}

//
$reportManager = new ReportManager();
$reportId = $reportManager->createReport('Sales Data', '模板1', ['source1',
'source2']);
$reportManager->modifyReport($reportId, 'Updated Sales Data', '模板2', ['source3']);
$reportDetails = $reportManager->viewReport($reportId);
$allReports = $reportManager->listReports();
$filteredReports = $reportManager->filterReports('Sales');
$reportPlugin = new ReportPlugin($reportManager);
$uploadSuccess = $reportPlugin->uploadReportData($reportId, ['data1']);
?>

<namespace org.mwyby.zzxob;
?php

require 'vendor/autoload.php';

use Predis\Client as RedisClient;

/**
 * Class SalesAnalysisController 主要负责销售分析记录的增、删、改、查等操作
 */
class SalesAnalysisController
{

```

```

private RedisClient $redisClient;

/**
 * SalesAnalysisController constructor.
 * @param RedisClient $redisClient Redis客户端实例用于和Redis交互
 */
public function __construct(RedisClient $redisClient)
{
    $this->redisClient = $redisClient;
}

/**
 * 增加销售分析记录
 * @param array $salesData 销售数据包含商品ID、名称等信息
 * @return bool
 */
public function addSalesAnalysis(array $salesData): bool
{
    $this->validateSalesData($salesData);

    $salesData['create_time'] = date('Y-m-d H:i:s');
    $this->saveToDatabase($salesData);

    $this->redisClient->publish('sales_analysis_created', json_encode($salesData));

    return true;
}

/**
 * 删除销售分析记录
 * @param int $salesId 分析记录ID
 * @return bool
 */
public function deleteSalesAnalysis(int $salesId): bool
{
    $this->checkRecordExists($salesId);

    $this->deleteFromDatabase($salesId);
    $this->redisClient->publish('sales_analysis_deleted', json_encode(['id' =>
$salesId]));

    return true;
}

/**
 * 编辑销售分析记录
 * @param int $salesId 分析记录ID
 * @param array $salesData 更新的销售数据
 * @return bool
 */
public function editSalesAnalysis(int $salesId, array $salesData): bool
{

```

```

        $this->checkRecordExists($salesId);
        $this->validateSalesData($salesData);

        $this->updateDatabaseRecord($salesId, $salesData);

        return true;
    }

    /**
     * 查看销售分析记录详细信息
     * @param int $salesId 分析记录ID
     * @return array
     */
    public function viewSalesAnalysis(int $salesId): array
    {
        return $this->fetchFromDatabase($salesId);
    }

    /**
     * 查询报表
     * @param array $filterCriteria 筛选条件
     * @return array
     */
    public function querySalesReport(array $filterCriteria): array
    {
        return $this->fetchSalesReport($filterCriteria);
    }

    /**
     * 校验销售数据的完整性
     * @param array $salesData 待校验的销售数据
     * @throws Exception 如果校验不通过则抛出异常
     */
    private function validateSalesData(array $salesData): void
    {
        if (empty($salesData['product_id']) || empty($salesData['sales_volume'])) {
            throw new Exception('产品ID和销售为必填项');
        }

        // 其他校验逻辑...
    }

    /**
     * 从数据库保存销售数据
     * @param array $salesData 销售数据
     */
    private function saveToDatabase(array $salesData): void
    {
        // 信息库插入逻辑...
    }

    /**
     * 检查销售记录是否存在

```

```

* @param int $salesId 分析记录ID
* @throws Exception 如果记录不存在则抛出异常
*/
private function checkRecordExists(int $salesId): void
{
    // 查询销售审计的逻辑，如果不存在则抛出异常...
}

/**
* 从数据库删除销售记录
* @param int $salesId 分析记录ID
*/
private function deleteFromDatabase(int $salesId): void
{
    // record库干掉逻辑...
}

/**
* 更新数据库记录
* @param int $salesId 分析记录ID
* @param array $salesData 更新的销售数据
*/
private function updateDatabaseRecord(int $salesId, array $salesData): void
{
    // 保存payload库逻辑...
}

/**
* 从数据库获取销售记录
* @param int $salesId 分析记录ID
* @return array 销售记录
*/
private function fetchFromDatabase(int $salesId): array
{
    // 查询销售log的逻辑，返回record...
    return [];
}

/**
* 根据筛选条件获取销售报表
* @param array $filterCriteria 筛选条件
* @return array 销售报表
*/
private function fetchSalesReport(array $filterCriteria): array
{
    // 根据筛选条件从data库查询销售报表...
    return [];
}
}

```

```
//
$redisClient = new RedisClient();
$salesController = new SalesAnalysisController($redisClient);
```

```
// 添加销售record
$salesController->addSalesAnalysis([
    'product_id' => 123456,
    'product_name' => '商品',
    'category_id' => 1,
    'category_name' => '类目',
    'time_granularity' => 'daily',
    'stats_time' => '2023-10-01 10:00:00',
    'sales_volume' => 100,
    'sales_amount' => 1000.00,
    'order_count' => 10,
    'user_count' => 8,
    'avg_price' => 100.00,
]);
```

// 这段业务需求并遵循给定的编码风格和要求。
<?php

// 实体：SalesData，用于表示销售record的基本结构

```
class SalesData
{
    // 属性定义：销售有效载荷的基本字段
    private $id;
    // 销售内容ID
    private $productId;
    // 产品ID
    private $quantity;
    // 销售
    private $salesAmount;
    // 销售金额
    private $saleDate;
    // 销售日期

    // 构造函数：设置销售record实体
    public function __construct($id, $productId, $quantity, $salesAmount, $saleDate)
    {
        $this->id = $id;
        $this->productId = $productId;
        $this->quantity = $quantity;
        $this->salesAmount = $salesAmount;
        $this->saleDate = $saleDate;
    }

    // 保存方法：使用活动record模式保存销售信息
    public function save()
    {
        // 此处trace库保存逻辑
        return true;
    }
}
```

```

// 返回保存success
}

// 读取销售内容的结构
public function getSalesInfo()
{
    // 返回销售信息数组
    return [
        'id' => $this->id,
        'productId' => $this->productId,
        'quantity' => $this->quantity,
        'salesAmount' => $this->salesAmount,
        'saleDate' => $this->saleDate
    ];
}

// 废弃销售消息体
public function delete()
{
    // 此处有效载荷库废弃逻辑
    return true;
    // 返回注销完成
}
}

// 值instance: SaleMetrics, 用于封装销售相关的度值
class SaleMetrics
{
    private $totalSales;
    // 总销售额
    private $totalQuantity;
    // 总销售

    public function __construct($totalSales, $totalQuantity)
    {
        $this->totalSales = $totalSales;
        $this->totalQuantity = $totalQuantity;
    }

    // 查询销售指标信息
    public function getMetrics()
    {
        return [
            'totalSales' => $this->totalSales,
            'totalQuantity' => $this->totalQuantity
        ];
    }
}

// 聚合根: SalesAggregate, 处理销售data的聚合逻辑
class SalesAggregate

```

```
{
    private $salesDataList;
    // 销售trace集合

    public function __construct()
    {
        $this->salesDataList = [];
    }

    public function addSaleData(SalesData $saleData)
    {
        // 将销售信息添加到集合中
        $this->salesDataList[] = $saleData;
    }

    // 计算销售指标
    public function calculateMetrics()
    {
        $totalSales = 0;
        $totalQuantity = 0;

        foreach ($this->salesDataList as $saleData) {
            $totalSales += $saleData->getSalesInfo()['salesAmount'];
            $totalQuantity += $saleData->getSalesInfo()['quantity'];
        }

        return new SaleMetrics($totalSales, $totalQuantity);
    }

    // 访问销售信息枚举
    public function getSalesDataList()
    {
        return $this->salesDataList;
    }
}

// 领域服务：SalesAnalysisService，处理销售分析逻辑
class SalesAnalysisService
{
    private $salesAggregate;

    // 构造函数装载销售聚合根
    public function __construct(SalesAggregate $salesAggregate)
    {
        $this->salesAggregate = $salesAggregate;
    }

    /**
     * 获取分析结果
     *
     * @param string $startDate 分析开始日期

```

```

* @param string $endDate 分析结束日期
* @return array 销售分析结果
*/
public function getAnalysisResults($startDate, $endDate)
{
    $filteredSalesData = array_filter($this->salesAggregate->getSalesDataList(),
function($saleData) use ($startDate, $endDate) {
        return ($saleData->getSalesInfo()['saleDate'] >= $startDate) && ($saleData-
>getSalesInfo()['saleDate'] <= $endDate);
    });

    // 计算过滤后的销售指标
    $metrics = $this->calculateMetrics($filteredSalesData);
    return $metrics->getMetrics();
}

// 基于销售信息计算销售指标
private function calculateMetrics(array $salesData)
{
    $totalSales = 0;
    $totalQuantity = 0;

    foreach ($salesData as $saleData) {
        $totalSales += $saleData->getSalesInfo()['salesAmount'];
        $totalQuantity += $saleData->getSalesInfo()['quantity'];
    }

    return new SaleMetrics($totalSales, $totalQuantity);
}

/**
 * 实时监控销售数据
 *
 * @return array 最新的销售数据
 */
public function monitorSalesData()
{
    // 这里应加入实时信息监控和变更逻辑
    // 如读取record库中的最新销售record
    return $this->salesAggregate->getSalesDataList();
}
}

// API资源类: SalesResource, 处理销售内容的序列化
class SalesResource
{
    public static function respond($salesData)
    {
        return json_encode([
            'data' => array_map(function($saleData) {

```

```

        return $saleData->getSalesInfo();
    }, $salesData),
]);
}
}

```

// 场景: initialize内容及分析

```

$salesAggregate = new SalesAggregate();
$salesAggregate->addSaleData(new SalesData(1, 101, 5, 500, '2023-10-01'));
$salesAggregate->addSaleData(new SalesData(2, 102, 3, 300, '2023-10-02'));
$salesAggregate->addSaleData(new SalesData(3, 101, 8, 800, '2023-10-03'));

```

```

$salesService = new SalesAnalysisService($salesAggregate);

```

// 查询特定时间段的销售分析结果

```

$analysisResults = $salesService->getAnalysisResults('2023-10-01', '2023-10-03');
echo SalesResource::respond($analysisResults);

```

?>

```

<namespace cn.xeuje.hxrtr;
?php

```

// Entity - UserBehavior

```

class UserBehavior {
    public function __construct(
        public string $id,
        public int $userId,
        public string $userName,
        public string $sessionId,
        public string $behaviorType,
        public int $productId,
        public string $productName,
        public int $categoryId,
        public string $categoryName,
        public DateTime $actionTime,
        public ?string $deviceType = null,
        public ?string $osVersion = null,
        public ?string $ipAddress = null,
        public ?string $province = null,
        public ?string $city = null,
        public ?string $extraInfo = null,
        public DateTime $createTime = new DateTime()
    ) {}
}

```

// Repository - UserBehaviorRepo

```

class UserBehaviorRepo {
    private $db;
    // Assume MongoDB Client

    public function __construct($dbClient) {

```

```

        $this->db = $dbClient->userBehaviorDB->userBehaviors;
    }

    public function add(UserBehavior $behavior): void {
        $this->db->insertOne($behavior);
    }

    public function delete(string $id): void {
        $this->db->deleteOne(['id' => $id]);
    }

    public function update(UserBehavior $behavior): void {
        $this->db->replaceOne(['id' => $behavior->id], $behavior);
    }

    public function get(string $id): ?UserBehavior {
        $data = $this->db->findOne(['id' => $id]);
        return $data ? new UserBehavior(...$data) : null;
    }

    public function getAll(): array {
        return array_map(fn($doc) => new UserBehavior(...$doc), $this->db->find()-
>toArray());
    }
}

// Service - UserBehaviorSVC
class UserBehaviorSVC {
    public function __construct(private UserBehaviorRepo $repo) {}

    public function createBehavior(array $data): string {
        $behavior = new UserBehavior(...array_merge($data, ['id' => $this-
>generateId()]));
        $this->repo->add($behavior);
        return $behavior->id;
    }

    public function removeBehavior(string $id): void {
        $this->repo->delete($id);
    }

    public function modifyBehavior(UserBehavior $behavior): void {
        $this->repo->update($behavior);
    }

    public function findBehavior(string $id): ?UserBehavior {
        return $this->repo->get($id);
    }
}

```

```

    public function listBehaviors(): array {
        return $this->repo->getAll();
    }

    private function generateId(): string {
        return 'UB-' . date('Ymd') . '-' . str_pad(rand(1, 9999), 4, '0',
STR_PAD_LEFT);
    }
}

// Controller - UserBehaviorController
class UserBehaviorController {
    public function __construct(private UserBehaviorSVC $svc) {}

    public function create(array $data): void {
        $this->svc->createBehavior($data);
    }

    public function delete(string $id): void {
        $this->svc->removeBehavior($id);
    }

    public function update(UserBehavior $behavior): void {
        $this->svc->modifyBehavior($behavior);
    }

    public function view(string $id): ?UserBehavior {
        return $this->svc->findBehavior($id);
    }

    public function index(): array {
        return $this->svc->listBehaviors();
    }
}

// Example usage
$dbClient = new MongoDB\Client("mongodb:
    //192.168.0.1:27017");
$repo = new UserBehaviorRepo($dbClient);
$svc = new UserBehaviorSVC($repo);
$controller = new UserBehaviorController($svc);

// Simulating API calls
$controller->create([
    'userId' => 123,
    'userName' => 'JohnDoe',
    'sessionId' => 'sess-xyz',
    'behaviorType' => 'click',
    'productId' => 456,
    'productName' => 'Product Name',
    'categoryId' => 789,

```

```

        'categoryName' => 'Category Name',
        'actionTime' => new DateTime(),
    ]);

```

```

$behaviorList = $controller->index();

```

```

$controller->delete('UB-20230905-0001');

```

```

$found = $controller->view('UB-20230905-0001');

```

```

?>
<?php

```

```

declare(strict_types=1);

```

```

use GraphQL\Type\Definition\Type;
use GraphQL\Type\Definition\ObjectType;
use GraphQL\Type\Definition\FieldDefinition;
use GraphQL\GraphQL;
use GraphQL\Schema;

```

```

class UserBehaviorLog
{

```

```

    // 信息库连接属性
    private PDO $db;

```

```

    // 构造函数，接受record库连接

```

```

    public function __construct(private PDO $database)
    {
        $this->db = $database;
        // 将传入消息体库连接赋值给属性
    }

```

```

/**

```

```

 * 获取用户为日志

```

```

 *

```

```

 * @param string $logType 日志类型，例如访问日志、点击日志、购买日志

```

```

 * @param string|null $startDate 开始日期（可空）

```

```

 * @param string|null $endDate 结束日期（可空）

```

```

 * @return array 返回指定类型的用户为日志

```

```

 *

```

```

 * 处理逻辑：

```

```

 * 1. 校验日志类型是否有效。

```

```

 * 2. 构建查询，根据类型和时间范围获取日志。

```

```

 */

```

```

    public function getUserBehaviorLogs(string $logType, ?string $startDate, ?string $endDate): array
    {

```

```

        if (!in_array($logType, ['access', 'click', 'purchase'])) {
            throw new InvalidArgumentException('Invalid log type provided.');
```

```

        // 严格校验日志类型
    }

```

```

$query = 'SELECT * FROM user_behavior_logs WHERE log_type = :logType';
$params = [':logType' => $logType];

if ($startDate) {
    $query .= ' AND log_date > :startDate';
    $params[':startDate'] = $startDate;
}

if ($endDate) {
    $query .= ' AND log_date <= :endDate';
    $params[':endDate'] = $endDate;
}

$stmt = $this->db->prepare($query);
// 使用准备好的语句防止SQL注入
$stmt->execute($params);
// 执查询

return $stmt->fetchAll(PDO::FETCH_ASSOC);
// 读取所有结果
}

/**
 * 导出用户为日志
 *
 * @param array $logs 日志数组
 * @param string $format 导出格式 (如csv, json)
 * @return string 返回导出文件路径
 *
 * 处理逻辑:
 * 1. 根据格式生成相应的文件。
 * 2. 确保文件保存路径的安全性和可写性。
 */
public function exportLogs(array $logs, string $format): string
{
    $filePath = './exports/user_logs.' . $format;
// 设置导出路径
    $allowedFormats = ['csv', 'json'];

    if (!in_array($format, $allowedFormats)) {
        throw new InvalidArgumentException('Unsupported export format.');
```

// 校验导出格式

```

    }

    if ($format === 'csv') {
        $fp = fopen($filePath, 'w');
        // 实例化CSV文件
        foreach ($logs as $log) {

```

```

        fputcsv($fp, $log);
    // 写入CSV
    }
    fclose($fp);
} elseif ($format === 'json') {
    file_put_contents($filePath, json_encode($logs));
// 写入JSON文件
}

return $filePath;
// 返回文件路径
}

/**
 * 删除指定类型的日志
 *
 * @param string $logType 日志类型
 * @return string 返回成功消息
 *
 * 处理逻辑：
 * 1. 确认日志类型的有效性。
 * 2. 对日志执删除操作，使用事务和悲观锁以防止写冲突。
 */
public function deleteLogs(string $logType): string
{
    if (!in_array($logType, ['access', 'click', 'purchase'])) {
        throw new InvalidArgumentException('Invalid log type provided.');
```

// 严格校验日志类型

```

    }

    $this->db->beginTransaction();
// 开始事务
    $stmt = $this->db->prepare('DELETE FROM user_behavior_logs WHERE log_type =
:logType FOR UPDATE');
    // 使用悲观锁
    $stmt->bindParam(':logType', $logType);
    $stmt->execute();
    $this->db->commit();
    // 提交事务

    return "Logs of type '{$logType}' have been deleted successfully.";
// 返回完成消息
}

/**
 * 创建GraphQL Schema
 *
 * @return Schema 返回构建的GraphQL Schema对象
 *
 * 处理逻辑：
 * 1. 定义GraphQL的查询及其解析函数。

```

```

* 2. 返回构建好的Schema。
*/
public function createSchema(): Schema
{
    $query = new ObjectType([
        'name' => 'Query',
        'fields' => [
            'logs' => [
                'type' => Type::listOf(Type::string()),
                'args' => [
                    'logType' => Type::string(),
                    'startDate' => Type::string(),
                    'endDate' => Type::string(),
                ],
                'resolve' => fn($root, $args) => $this->
>getUserBehaviorLogs($args['logType'], $args['startDate'] ?? null, $args['endDate'] ??
null)
            ],
        ],
    ]);

    // 构建并返回GraphQL的Schema
    return new Schema([
        'query' => $query,
    ]);
}

/**
 * 处理GraphQL请求
 *
 * @param string $query GraphQL查询串
 * @return array 返回查询结果
 *
 * 处理逻辑：
 * 1. 建立GraphQL请求上下文。
 * 2. 进查询处理并返回结果。
 */
public function handleGraphQLRequest(string $query): array
{
    $schema = $this->createSchema();
    // createSchema
    $result = GraphQL::executeQuery($schema, $query);
    // 执查询

    return $result->toArray();
    // 返回查询结果
}

}

// 内容库连接（使用相应的连接信息）
$database = new PDO('mysql:host=192.168.0.1;dbname=your_db', 'username', 'password');
$database->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
// 设置异常模式

```

```
// 实例化客户端为日志管理类
$userBehaviorLogManager = new UserBehaviorLog($database);

// Example usage:
$query = ' { logs(logType: "access") } ';
    // GraphQL查询
$response = $userBehaviorLogManager->handleGraphQLRequest($query);
// 处理GraphQL调用
echo json_encode($response);
// 输出结果
?>
```